

Docker Fundamentals

Overview

Docker is a containerization platform that packages an application along with all its dependencies into a **container**. Containers provide a consistent runtime environment, ensuring the application behaves the same on every machine.

Why Do We Need Docker?

Scenario

You developed a web application using:

- Node.js
- HTML
- CSS
- JavaScript

The application works perfectly on your laptop.

Question: Will it run the same on another developer's laptop?

Answer: Maybe.

The application may fail because of:

- Different Node.js versions
- Missing dependencies
- Different operating system
- Missing environment variables
- Different package versions

Docker solves this problem by packaging the application and its dependencies together.

Build Once → Run Anywhere

Virtual Machines vs Containers

Virtual Machine	Docker Container
Includes complete Guest OS	Shares Host OS Kernel
Heavy	Lightweight
Slow startup	Starts in seconds

Virtual Machine	Docker Container
High resource usage	Low resource usage
Large image size	Small image size
Hardware Virtualization	OS-level Virtualization

What is Docker?

Docker is a platform used to package an application along with its dependencies into a portable container.

Think of it like a **shipping container**.

Just as a shipping container carries goods safely across the world, Docker containers carry your application and everything it needs to run.

A Docker container may include:

- Application Code
 - Libraries
 - Runtime
 - Environment Variables
 - Dependencies
 - Configuration Files
-

Docker Image vs Docker Container

Docker Image

A Docker Image is a **blueprint** or **template** used to create containers.

It contains:

- Application
- Runtime
- Dependencies
- Libraries
- Configuration

Example:

```
ubuntu  
nginx
```

```
python:3.12
node:20
```

Docker Container

A Docker Container is a **running instance** of a Docker Image.

Example:

```
Image
  ↓
docker run
  ↓
Container
```

Real-life analogy:

Class → Object

Image → Container

Docker Workflow

```
Write Application
  ↓
Create Dockerfile
  ↓
Build Image
  ↓
Run Container
  ↓
Deploy Anywhere
```

Lab 1 – Install Docker on Ubuntu / EC2

Official Documentation:

<https://docs.docker.com/engine/install/ubuntu/>

Step 1 – Update Packages

```
sudo apt update
sudo apt install ca-certificates curl
```

Step 2 – Add Docker Repository

```
sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Update package list:

```
sudo apt update
```

Step 3 – Install Docker

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

Step 4 – Verify Installation

```
docker --version
```

```
sudo systemctl status docker
```

Run Docker Without sudo

```
sudo usermod -aG docker ubuntu
```

```
newgrp docker
```

```
sudo systemctl daemon-reload
```

Verify:

```
docker run hello-world
```

Lab 2 – First Docker Container

Run:

```
docker run hello-world
```

What happens?

- Docker checks for the image locally.
 - If not available, it downloads it from Docker Hub.
 - Creates a new container.
 - Executes the application.
 - Exits automatically.
-

Three Modes of Running Containers

1. Foreground Mode

```
docker run hello-world
```

or

```
docker run ubuntu
```

Characteristics:

- Runs in current terminal
 - Displays logs
 - Terminal remains occupied
 - Stops when application exits
-

2. Detached Mode

Runs container in background.

```
docker run -d nginx
```

Interactive Ubuntu:

```
docker run -dt ubuntu
```

Useful Commands

```
docker ps
```

```
docker logs <container>
```

```
docker stop <container>
```

3. Interactive Mode

Launch container and immediately enter shell.

```
docker run -it ubuntu bash
```

Exit container:

```
Ctrl + P + Q
```

(keeps container running)

To stop:

```
exit
```

Reconnect:

```
docker attach <container-id>
```

or

```
docker exec -it <container-id> bash
```

`docker exec` is generally preferred because it opens a new shell without affecting the main container process.

Frequently Used Docker Commands

Images

```
docker pull nginx
```

```
docker images
```

```
docker rmi image-name
```

Containers

```
docker run nginx
```

```
docker ps
```

```
docker ps -a
```

```
docker ps -l
```

```
docker ps -q
```

```
docker stop container
```

```
docker start container
```

```
docker restart container
```

```
docker rm container
```

System Commands

```
docker system df
```

```
docker system prune
```

```
docker stats
```

Lab 3 – Inspect Docker Objects

The `docker inspect` command displays detailed JSON metadata about Docker objects.

Inspect a container:

```
docker inspect <container-id>
```

Inspect an image:

```
docker inspect nginx
```

Useful examples:

Container IP Address

```
docker inspect -f '{{.NetworkSettings.IPAddress}}' <container>
```

Container Name

```
docker inspect -f '{{.Name}}' <container>
```

Container Status

```
docker inspect -f '{{.State.Status}}' <container>
```

Mounted Volumes

```
docker inspect -f '{{json .Mounts}}' <container>
```

Environment Variables

```
docker inspect -f '{{json .Config.Env}}' <container>
```

Useful Commands

View Logs

```
docker logs <container>
```

Live Logs

```
docker logs -f <container>
```

Processes Running

```
docker top <container>
```

Copy Files

```
docker cp test.txt container:/tmp
```

Rename Container

```
docker rename old-name new-name
```

Container Resource Usage

Cgroups (Control Groups)

Overview

Cgroups (Control Groups) are a Linux kernel feature that allows you to limit and monitor the resources consumed by processes. Docker uses cgroups to ensure that containers do not consume excessive CPU or memory.

Example

Suppose your server has:

- 2 vCPUs
- 4 GB RAM
- 20 GB Storage

Running three containers:

- Frontend
- Backend
- Database

If one container consumes all the memory or CPU, the other containers may become slow or even crash. Cgroups help prevent this by enforcing resource limits.

Limit Memory

Run a container with a maximum memory limit of 512 MB.

```
docker run -d --memory="512m" ubuntu sleep infinity
```

Verify the limit:

```
docker inspect <container-name>
```

Limit CPU

Allow the container to use only half of one CPU core.

```
docker run -d --cpus="0.5" ubuntu sleep infinity
```

Limit Number of Processes

```
docker run -d --pids-limit=100 ubuntu sleep infinity
```

Update Resources of a Running Container

Increase memory:

```
docker update --memory="1g" <container-name>
```

Update CPU limit:

```
docker update --cpus="1" <container-name>
```

Check live resource usage:

```
docker stats
```

Dockerfile

A **Dockerfile** is a text file containing instructions used to build a Docker Image.

Example:

```
FROM python:3.12-trixie
WORKDIR /app
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 8080
CMD ["python3", "app.py"]
```

Build the Image

```
docker build -t demo-image:v1 .
```

Verify:

```
docker images
```

Lab 4 – Run Your Image

Start a container from the image:

```
docker run -d -p 8080:8080 --name demo demo-image:v1
```

Verify:

```
docker ps
```

Open the application:

```
http://<Server-IP>:8080
```

Stop:

```
docker stop demo
```

Remove:

```
docker rm demo
```

Create an Image from a Running Container

Sometimes changes are made manually inside a container.

Save those changes as a new image:

```
docker commit <container-id> myimage:v1
```

Verify:

```
docker images
```

Docker Compose

Docker Compose helps run multiple containers using a single YAML file.

Start all services:

```
docker compose up
```

Start in detached mode:

```
docker compose up -d
```

Stop services:

```
docker compose down
```

View running services:

```
docker compose ps
```

Push Images to Docker Hub

Step 1 – Login

```
docker login
```

Step 2 – Tag the Image

```
docker tag demo-image:v1 <dockerhub-username>/demo-image:v1
```

Example:

```
docker tag demo-image:v1 ajit0101/demo-image:v1
```

Step 3 – Push the Image

```
docker push ajit0101/demo-image:v1
```

Anyone can now pull the image using:

```
docker pull ajit0101/demo-image:v1
```

Additional Practice Labs

Lab 5 – Run Nginx

```
docker run -d --name nginx-server -p 80:80 nginx
```

Verify:

```
docker ps
```

Open:

```
http://<Server-IP>
```

Lab 6 – Port Mapping

Run Apache on port 8080:

```
docker run -d -p 8080:80 httpd
```

Open:

```
http://<Server-IP>:8080
```

Lab 7 – Execute Commands Inside a Running Container

```
docker exec -it nginx-server bash
```

If bash is unavailable:

```
docker exec -it nginx-server sh
```

Check hostname:

```
hostname
```

Exit:

```
exit
```

Lab 8 – View Container Logs

Generate logs:

```
docker logs nginx-server
```

Follow logs in real time:

```
docker logs -f nginx-server
```

Lab 9 – Clean Up Docker Resources

Remove stopped containers:

```
docker container prune
```

Remove unused images:

```
docker image prune
```

Remove everything unused:

```
docker system prune -a
```

Docker Workflow Summary

Write Application



Create Dockerfile



Build Image
docker build



Run Container
docker run



Test Application



Tag Image
docker tag



Push to Docker Hub

docker push



Deploy Anywhere

Common Docker Commands

```
docker --version
docker images
docker pull <image>
docker build -t image-name .
docker run -d image-name
docker ps
docker ps -a
docker logs <container>
docker exec -it <container> bash
docker inspect <container>
docker stats
docker stop <container>
docker start <container>
docker restart <container>
docker rm <container>
docker rmi <image>
docker commit <container> image-name
docker compose up -d
docker compose down
docker login
docker tag
docker push
docker system prune -a
```

Key Takeaways

- Docker packages applications with all dependencies.
- Images are templates; containers are running instances of images.
- Containers are lightweight compared to virtual machines.
- Use **Dockerfiles** to build custom images.
- Use **Docker Compose** for multi-container applications.
- Use **docker inspect** to view container metadata.
- Use **cgroups** to control CPU and memory usage.
- Push images to **Docker Hub** for sharing and deployment.